

Project Report

Cancer Classification - Python Essentials Project

1. Title

Cancer Classification Using Python

2. Introduction

This project demonstrates the use of Python programming fundamentals by creating a simple command-line application for educational risk classification. The application accepts basic user inputs and applies predefined conditional rules to calculate a score.

3. Objective

The main objectives are:

- To practice Python functions.
- To understand variables and data types.
- To use conditional statements.
- To implement loops and input validation.
- To demonstrate exception handling.
- To create a command-line executable Python project.

4. Problem Statement

Create a Python program that accepts basic patient information and selected symptoms, calculates a predefined score, and displays an educational risk classification.

5. Methodology

The application follows these steps:

1. Display the project introduction.
2. Collect the patient's name and age.
3. Collect tumor size.
4. Collect three yes/no symptom responses.
5. Calculate a score using predefined rules.
6. Compare the score with classification thresholds.
7. Display the calculated score and classification.

6. Algorithm

1. Start the program.
2. Read patient name.
3. Read and validate age.

4. Read and validate tumor size.
5. Read pain, lump, and weight-loss responses.
6. Initialize score to zero.
7. Add points according to tumor-size rules.
8. Add symptom points.
9. If score ≥ 6 , classify as High Risk.
10. Else if score ≥ 3 , classify as Medium Risk.
11. Otherwise classify as Low Risk.
12. Display the result.
13. End.

7. Python Concepts Used

- Functions
- Variables
- Strings
- Integers and floating-point numbers
- `if`, `elif`, and `else`
- `while` loops
- `try` and `except`
- User input
- Formatted console output
- Main-function execution pattern

8. Project Files

main.py

Contains the complete application logic, including data collection, validation, score calculation, and result display.

README.md

Contains project setup, execution instructions, classification logic, and project information.

requirements.txt

Documents that no external packages are required.

9. Testing

The following types of tests should be performed:

Test	Purpose
Valid patient details	Verify normal execution

Test	Purpose
Invalid age	Verify age validation
Negative tumor size	Verify tumor-size validation
Invalid yes/no response	Verify response validation
Different tumor sizes	Verify scoring rules
Different symptom combinations	Verify final classification

10. Screenshots / Results

10.1 Terminal Result

The following screenshot shows a sample execution of the application with patient input and the resulting educational classification.

```

Problems  Output  Debug Console  Terminal  Ports

Enter patient details
-----
Name: Rahul
Age: 18
Tumor size in cm: 2
Pain (yes/no): no
Lump (yes/no): no
Weight loss (yes/no): no

=====
          CANCER CLASSIFICATION
=====
Name       : Rahul
Age        : 18
Tumor size : 2.0 cm
Score      : 2
Result     : Low Risk
=====
Educational purpose only.
This result is not a medical diagnosis.
Save this result to results.txt? (yes/no): no

```

10.2 Project Files

The following screenshot shows the project files in the development environment, including statement.md and PROJECT_REPORT.md.

README.md	×
statement.md	×
PROJECT_REPORT.md	×
.gitignore	×
tempCodeRunnerFile.py	×
main.py	×
__init__.py	×
classifier.py	×
input_handler.py	×
result_report.py	×
validator.py	×

11. Expected Output

The application displays:

- Patient name
- Age
- Tumor size
- Calculated score
- Educational risk classification

12. Limitations

This is a basic rule-based educational program. It does not use a medical dataset, machine-learning model, laboratory results, medical history, imaging information, or clinically validated diagnostic criteria.

Therefore, the classification cannot be used to diagnose cancer or determine an individual's actual medical risk.

13. Conclusion

The project demonstrates how Python fundamentals can be combined to create a functional command-line application. It provides practical experience with functions, conditional logic, loops, validation, exception handling, and user interaction.

14. Future Improvements

Possible educational extensions include:

- Storing records in a file or database.
- Adding automated unit tests.
- Creating a menu-driven interface.

- Adding data visualization.
- Using a properly validated public dataset for a separate machine-learning study.
- Adding more comprehensive input validation.